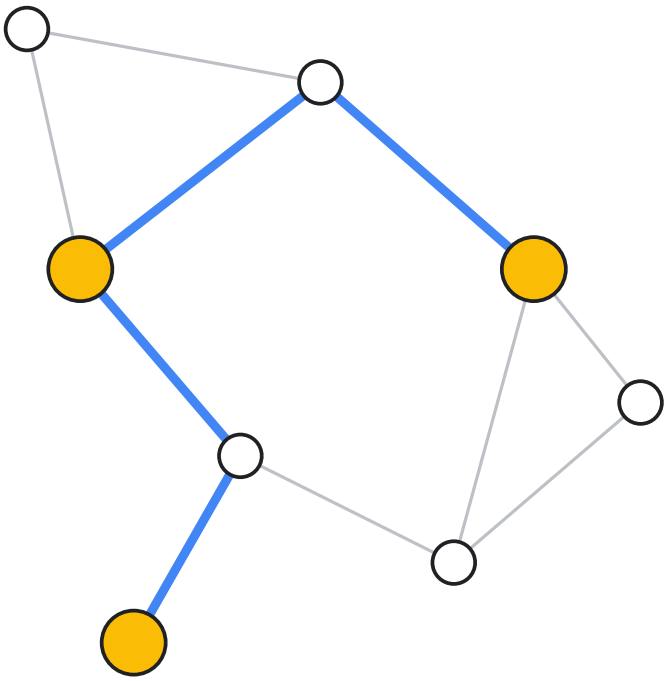


Lightpath

Latency-aware cloud task placement

Demo notebook companion: theory, code, outputs and diagrams

Tanishq Hanumanta
Sushil Pal
Suhaan Kapoor
Arjun Pal
Om Pilaji

Guided by Dr. Monday Jubrin Abdullahi
Woxsen University

Generated from lightpath_demo_standalone.ipynb (executed; all ten self-checks pass).

Contents

Overview 3

The problem in one picture . . . 3

Notation . . . 4

How to run the notebook . . . 4

Setup and the mini-library 5

Design choices worth knowing . . . 5

1. The campus network 9

2. Backbone: Prim and Kruskal agree 10

3. Shortest paths from the Main Entrance (Dijkstra) 11

4. Negative transfer costs: Bellman-Ford and Johnson 12

5. All-pairs latency: the automatic choice 14

6. Knapsack DP: choosing servers within a budget 16

7. Full placement on India and Nigeria 17

Capacity-aware assignment . . . 17

Heuristics . . . 17

Exact optimum with a MILP . . . 17

8. The India plan on a map 20

9. Experiment highlights 21

10. Self-checks 22

Appendix 23

A. Complexity summary . . . 23

B. Glossary . . . 23

C. Where to change things . . . 23

Overview

Servers are graph nodes and link latencies are edge weights. **Lightpath** decides where to host service replicas (or VMs) so that users see the lowest **demand-weighted average latency**, while every replica stays within its CPU and memory capacity and the total hosting cost stays within a budget. This document walks through the demo notebook step by step: the **theory** behind each algorithm, the **code**, the real **outputs** it produced, and **diagrams** that make each idea visible.

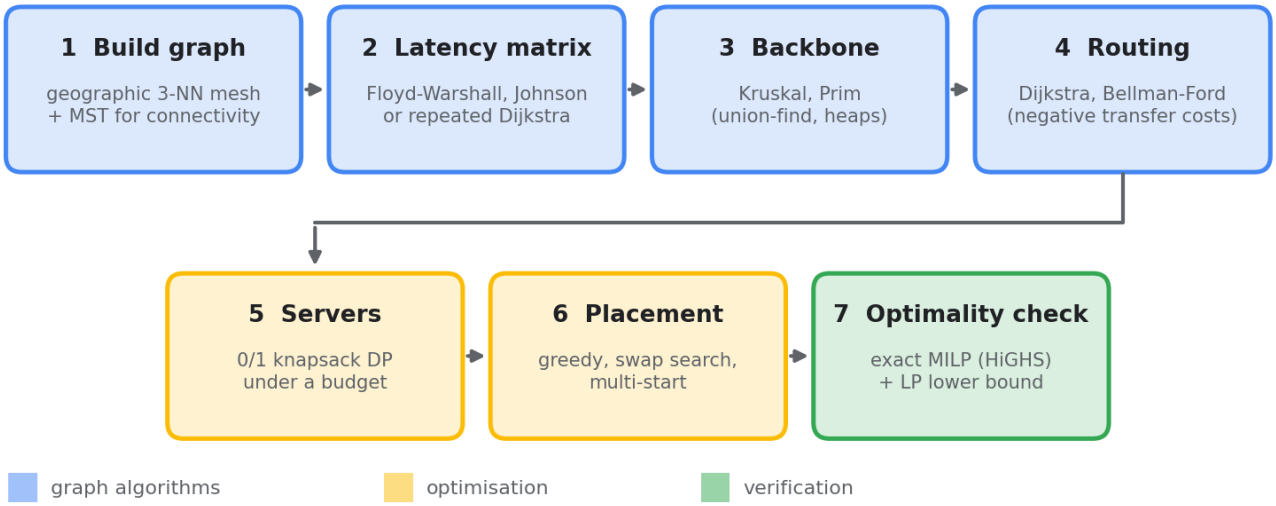


Figure 1. The Lightpath pipeline. Each stage has a section in this document and a cell in the notebook.

The problem in one picture

Given a network, the demand each city generates, a capacity and cost for every possible host, choose at most K sites to host replicas and decide which replica serves which city.

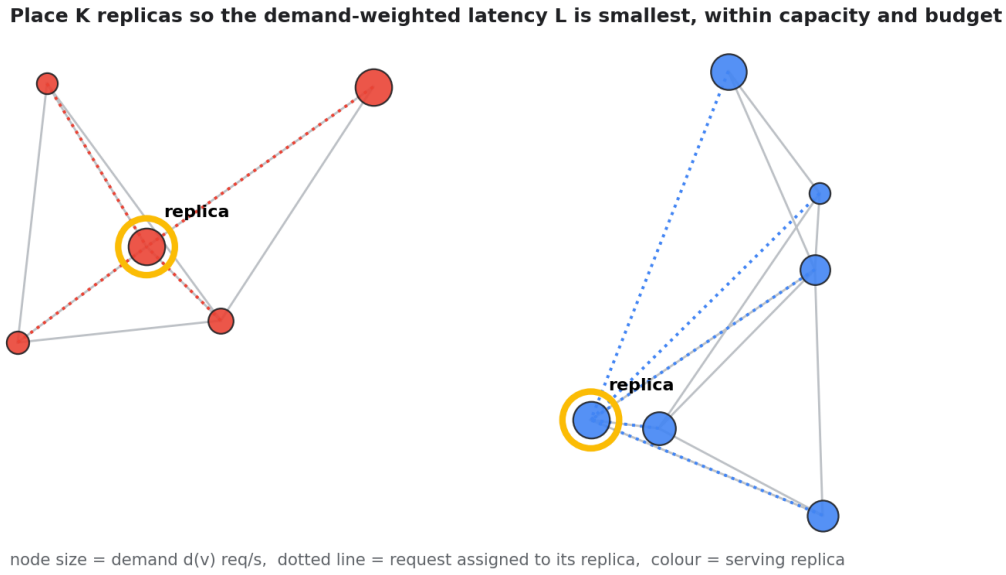


Figure 2. Replica placement. Larger nodes generate more demand; dotted lines show each node being served by its nearest replica that still has capacity.

The quantity we minimise is the demand-weighted average latency over all served demand:

$$L(R) = \frac{\sum_v d_v \cdot \mathrm{dist}(v,\, r(v))}{\sum_v d_v}$$

Here d_v is the demand (requests per second) at node *v*, *r(v)* is the replica serving it, and dist is the shortest-path latency. Heavy-demand cities therefore count more than quiet ones.

Notation

Symbol	Meaning	In the code
G = (V, E)	Undirected network: sites and latency-weighted links	`Graph`
w(u,v)	Link latency in ms (always positive)	`g.adj[u]`
dist(u,v)	Shortest-path latency between two sites	`I.D`, from `apsp`
d_v	Demand generated at node v (req/s)	`I.dem`
cap_j	Capacity of site j in demand units (req/s)	`I.capu`
K, B	Maximum number of replicas, hosting budget	`I.K`, `I.budget`
R	Chosen replica sites	`r["R"]`

Honest scope. City coordinates are real. Demands, capacities, hosting costs and the campus layout are illustrative inputs chosen for the demo, so the numbers show how the method behaves rather than measure a real deployment. The campus layout in particular is a placeholder for the official campus map.

How to run the notebook

```
pip install numpy scipy pandas matplotlib jupyter
jupyter notebook lightpath_demo_standalone.ipynb   # then Kernel > Restart & Run All
```

The notebook needs only NumPy, SciPy, pandas and matplotlib. Every algorithm is written from scratch in one "mini-library" cell and then cross-checked against a second method, so each result can be explained in a viva.

Setup and the mini-library

The first four code cells prepare everything the later sections use. They are shown in full so the document is complete on its own.

Cell	What it defines	Why it matters
1	Imports, the Google colour palette, plot style	One consistent look for every figure
2	Graph, UnionFind, indexed heap, Kruskal, Prim, Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson, automatic APSP, knapsack DP	All graph algorithms, written from scratch
3	Haversine distance, latency model, geographic mesh builder, city and campus data, transfer-cost graph	Turns coordinates into a connected, weighted graph
4	Instance, capacity-aware assignment, greedy, local search, multi-start, random baseline, exact MILP, strategy runner	The placement engine and its optimality check

Design choices worth knowing

Adjacency lists. The graph stores, for every node, the list of its neighbours and link weights. An undirected link is stored in both directions. This uses space proportional to V + E and lets Dijkstra and Prim scan only real neighbours, which is what makes them fast on sparse networks.

Union-find with path compression. Kruskal must ask "are these two sites already connected?" many times. Union-find answers in near-constant amortised time by pointing every visited node closer to its root.

Indexed binary heap. Prim's eager variant needs *decrease-key*: lowering a node's priority when a cheaper link appears. The heap keeps a position map so any entry can be found and moved up in O(log V). The lazy variant instead pushes duplicates into Python's heapq and skips stale entries when they surface.

Code cell 1: imports and palette

```
import time, math, heapq, random
from collections import namedtuple
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import milp, LinearConstraint, Bounds
pd.set_option('display.width', 140)
BLUE, RED, YEL, GRN, INK, GREY = '#4285F4', '#EA4335', '#FBBC05', '#34A853', '#202124', '#5F6368'
plt.rcParams.update({'axes.spines.top': False, 'axes.spines.right': False, 'figure.dpi': 110})
```

Code cell 2: mini-library (graph, heaps, MST, shortest paths, knapsack)

```
# ---------------- mini-library: graph, heaps, MST, shortest paths, knapsack ----------------
class NegativeCycleError(Exception): pass
MST = namedtuple('MST', 'edges total_weight')
Knap = namedtuple('Knap', 'chosen value weight')

class Graph:
    def __init__(s, nodes, edges, directed=False):
        s.nodes, s.n, s.directed, s.edges, s.adj = nodes, len(nodes), directed, [], [[] for _ in nodes]
        for u, v, w in edges: s.add(u, v, w)
    def add(s, u, v, w):
        s.edges.append((u, v, w)); s.adj[u].append((v, w))
        if not s.directed: s.adj[v].append((u, w))
    def density(s): return len(s.edges) / (s.n * (s.n - 1) / (1 if s.directed else 2))
    def summary(s): return f"{s.n} nodes, {len(s.edges)} {'arcs' if s.directed else 'links'}, density {s.density():.3f}"

class UnionFind:
    def __init__(s, n): s.p = list(range(n))
    def find(s, x):
        while s.p[x] != x: s.p[x] = s.p[s.p[x]]; x = s.p[x]          # path compression
        return x
    def union(s, a, b):
        a, b = s.find(a), s.find(b)
        if a == b: return False
        s.p[a] = b; return True

class IdxHeap:                                                      # binary heap with decrease-key
    def __init__(s): s.a, s.pos = [], {}
    def _sw(s, i, j): s.a[i], s.a[j] = s.a[j], s.a[i]; s.pos[s.a[i][1]] = i; s.pos[s.a[j][1]] = j
    def _up(s, i):
        while i and s.a[i][0] < s.a[(i-1)//2][0]: s._sw(i, (i-1)//2); i = (i-1)//2
    def _down(s, i):
        while True:
            l, r, m = 2*i+1, 2*i+2, i
            if l < len(s.a) and s.a[l][0] < s.a[m][0]: m = l
```

```
            if r < len(s.a) and s.a[r][0] < s.a[m][0]: m = r
            if m == i: return
            s._sw(i, m); i = m
    def push(s, k, key): s.a.append([key, k]); s.pos[k] = len(s.a)-1; s._up(len(s.a)-1)
    def decrease(s, k, key):
        i = s.pos[k]
        if key < s.a[i][0]: s.a[i][0] = key; s._up(i)
    def pop(s):
        top, last = s.a[0], s.a.pop()
        if s.a: s.a[0] = last; s.pos[last[1]] = 0; s._down(0)
        del s.pos[top[1]]; return top[1], top[0]
    def __contains__(s, k): return k in s.pos
    def __bool__(s): return bool(s.a)

def kruskal(g):                                                 # O(E log E)
    uf, T, tot = UnionFind(g.n), [], 0.0
    for u, v, w in sorted(g.edges, key=lambda e: e[2]):
        if uf.union(u, v): T.append((u, v, w)); tot += w
    return MST(T, tot)

def prim(g, heap='lazy'):                                       # O(E log V)
    inT, par, best, T, tot = [False]*g.n, [-1]*g.n, [math.inf]*g.n, [], 0.0
    if heap == 'lazy':                                          # heapq + lazy deletion
        pq = [(0.0, 0, -1)]
        while pq:
            w, u, p = heapq.heappop(pq)
            if inT[u]: continue
            inT[u] = True
            if p >= 0: T.append((p, u, w)); tot += w
            for v, wv in g.adj[u]:
                if not inT[v]: heapq.heappush(pq, (wv, v, u))
    else:                                                       # eager decrease-key
        h = IdxHeap(); best[0] = 0; h.push(0, 0)
        while h:
            u, w = h.pop(); inT[u] = True
            if par[u] >= 0: T.append((par[u], u, w)); tot += w
            for v, wv in g.adj[u]:
                if inT[v]: continue
                if v not in h: best[v], par[v] = wv, u; h.push(v, wv)
                elif wv < best[v]: best[v], par[v] = wv, u; h.decrease(v, wv)
    return MST(T, tot)

class SP:
    def __init__(s, dist, prev, src): s.dist, s.prev, s.src = dist, prev, src
    def path_to(s, t):
        p = [t]
        while p[-1] != s.src: p.append(s.prev[p[-1]])
        return p[::-1]

def dijkstra(g, s, weight=None):                                # O(E log V), needs w >= 0
    dist, prev = [math.inf]*g.n, {}; dist[s] = 0.0; pq = [(0.0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, w in g.adj[u]:
            if w < 0: raise ValueError(f'negative arc {u}->{v} ({w:.2f}): Dijkstra needs w >= 0')
            if d + w < dist[v]: dist[v] = d + w; prev[v] = u; heapq.heappush(pq, (dist[v], v))
    return SP(dist, prev, s)

def bellman_ford(g, s):                                         # O(V E), handles negative arcs
    dist, prev = [math.inf]*g.n, {}; dist[s] = 0.0
    for _ in range(g.n - 1):
        ch = False
        for u in range(g.n):
            for v, w in g.adj[u]:
                if dist[u] + w < dist[v] - 1e-12: dist[v] = dist[u] + w; prev[v] = u; ch = True
        if not ch: break
    for u in range(g.n):
        for v, w in g.adj[u]:
            if dist[u] + w < dist[v] - 1e-9: raise NegativeCycleError('negative cycle reachable')
    return SP(dist, prev, s)

def floyd_warshall_numpy(g):                                    # Theta(V^3), vectorised
    D = np.full((g.n, g.n), np.inf); np.fill_diagonal(D, 0)
    for u in range(g.n):
        for v, w in g.adj[u]: D[u, v] = min(D[u, v], w)
    for k in range(g.n): D = np.minimum(D, D[:, [k]] + D[[k], :])
    return D

def johnson(g):                                                 # O(V E log V)
    aug = Graph(g.nodes + [{}], [(u, v, w) for u in range(g.n) for v, w in g.adj[u]] + [(g.n, v, 0.0) for v in range(g.n)],
      ↪ directed=True)
    h = bellman_ford(aug, g.n).dist                             # potentials from a virtual source
    rw = Graph(g.nodes, [(u, v, w + h[u] - h[v]) for u in range(g.n) for v, w in g.adj[u]], directed=True)
    D = np.array([[d - h[s] + h[t] for t, d in enumerate(dijkstra(rw, s).dist)] for s in range(g.n)])
    return D

APSP = namedtuple('APSP', 'dist algorithm')
def apsp(g):                                                    # automatic choice
    if any(w < 0 for u in range(g.n) for _, w in g.adj[u]): return APSP(johnson(g), 'Johnson')
    if g.density() > 0.25: return APSP(floyd_warshall_numpy(g), 'Floyd-Warshall (numpy)')
    return APSP(np.array([dijkstra(g, s).dist for s in range(g.n)]), 'repeated Dijkstra')

def knapsack_01(vals, wts, W):                                  # Theta(nW)
    n = len(vals); T = [[0]*(W+1) for _ in range(n+1)]
```

```
    for i in range(1, n+1):
        for c in range(W+1):
            T[i][c] = T[i-1][c]
            if wts[i-1] <= c: T[i][c] = max(T[i][c], T[i-1][c-wts[i-1]] + vals[i-1])
    c, chosen = W, []
    for i in range(n, 0, -1):
        if T[i][c] != T[i-1][c]: chosen.append(i-1); c -= wts[i-1]
    return Knap(sorted(chosen), T[n][W], sum(wts[i] for i in chosen))
```

Code cell 3: datasets, geographic mesh, latency and transfer-cost models

```
# ---------------- datasets: geographic mesh, latency model, placement instances ----------------
def great_circle_km(a, b):
    p = math.pi/180; h = math.sin((b['y']-a['y'])*p/2)**2 +
      ↪ math.cos(a['y']*p)*math.cos(b['y']*p)*math.sin((b['x']-a['x'])*p/2)**2
    return 12742*math.asin(math.sqrt(h))
latency = lambda km: 1.3*km/200 + 0.3                                  # ms, one way

def geo_graph(nodes, km_fn):                                           # 3-nearest-neighbour mesh + MST so it is connected
    n = len(nodes); lat = lambda i, j: round(latency(km_fn(nodes[i], nodes[j])), 4)
    full = Graph(nodes, [(i, j, lat(i, j)) for i in range(n) for j in range(i+1, n)])
    E = {(min(u, v), max(u, v)): w for u, v, w in kruskal(full).edges}
    for i in range(n):
        for j in sorted((j for j in range(n) if j != i), key=lambda j: km_fn(nodes[i], nodes[j]))[:3]: E[(min(i, j), max(i,
          ↪ j))] = lat(i, j)
    return Graph(nodes, [(u, v, w) for (u, v), w in sorted(E.items())])

# name, lat, lon, demand (req/s), tier
INDIA = [('Delhi',28.61,77.21,40,'core'),('Mumbai',19.08,72.88,45,'core'),('Bengaluru',12.97,77.59,38,'core'),('Chennai',13.08,8
  ↪ 0.27,28,'core'),
 ('Kolkata',22.57,88.36,30,'edge'),('Hyderabad',17.38,78.49,30,'edge'),('Pune',18.52,73.86,20,'edge'),('Ahmedabad',23.02,72.57,1
  ↪ 8,'edge'),
 ('Jaipur',26.91,75.79,12,'edge'),('Lucknow',26.85,80.95,12,'edge'),('Kanpur',26.45,80.35,10,'edge'),('Nagpur',21.15,79.09,10,'e
  ↪ dge'),
 ('Indore',22.72,75.86,12,'edge'),('Bhopal',23.26,77.41,8,'edge'),('Visakhapatnam',17.69,83.22,9,'edge'),('Patna',25.59,85.14,10
  ↪ ,'edge'),
 ('Vadodara',22.31,73.18,8,'edge'),('Surat',21.17,72.83,14,'edge'),('Coimbatore',11.02,76.96,9,'edge'),('Kochi',9.93,76.27,8,'ed
  ↪ ge'),
 ('Chandigarh',30.73,76.78,8,'edge'),('Guwahati',26.14,91.74,6,'edge'),('Bhubaneswar',20.30,85.82,7,'edge'),('Thiruvananthapuram
  ↪ ',8.52,76.94,5,'edge'),
 ('Woxsen University',17.63,78.07,12,'edge')]
NIGERIA = [('Lagos',6.52,3.38,60,'core'),('Abuja',9.07,7.40,40,'core'),('Port
  ↪ Harcourt',4.82,7.05,30,'core'),('Kano',12.00,8.52,28,'edge'),
 ('Ibadan',7.38,3.95,22,'edge'),('Benin
  ↪ City',6.34,5.63,16,'edge'),('Kaduna',10.52,7.44,14,'edge'),('Maiduguri',11.83,13.15,10,'edge'),
 ('Zaria',11.11,7.72,8,'edge'),('Aba',5.12,7.37,12,'edge'),('Jos',9.93,8.89,10,'edge'),('Ilorin',8.50,4.55,10,'edge'),
 ('Oyo',7.85,3.93,6,'edge'),('Enugu',6.44,7.50,14,'edge'),('Abeokuta',7.16,3.35,8,'edge'),('Onitsha',6.14,6.79,10,'edge'),
 ('Warri',5.52,5.75,9,'edge'),('Sokoto',13.06,5.24,6,'edge'),('Calabar',4.95,8.33,7,'edge'),('Katsina',12.99,7.60,5,'edge'),
 ('Akure',7.25,5.19,6,'edge'),('Bauchi',10.31,9.84,5,'edge'),('Makurdi',7.73,8.52,5,'edge'),('Owerri',5.48,7.03,8,'edge')]
# illustrative campus layout in metres, west to east (replace with the official map coordinates)
CAMPUS = [('Main Entrance (gate)',0,200,'edge'),('Admin Block',120,180,'core'),('Library',200,230,'edge'),('Academic Block
  ↪ A',260,150,'edge'),
 ('Academic Block B',320,200,'edge'),('Data Centre',330,40,'core'),('Auditorium',380,120,'edge'),('Student
  ↪ Centre',440,210,'edge'),
 ('Innovation Lab',450,60,'edge'),('Hostel North',520,300,'edge'),('Hostel South',520,90,'edge'),('Sports
  ↪ Complex',640,160,'edge')]

def region_graph(rows):
    nodes = [dict(name=r[0], y=r[1], x=r[2], demand=r[3], tier=r[4], cpu=64 if r[4] == 'core' else 32) for r in rows]
    return geo_graph(nodes, great_circle_km)
def campus_graph():
    nodes = [dict(name=r[0], x=r[1], y=r[2], tier=r[3], cpu=64 if r[3] == 'core' else 32, demand=10) for r in CAMPUS]
    return geo_graph(nodes, lambda a, b: math.hypot(a['x']-b['x'], a['y']-b['y'])/1000)

def transfer_cost_graph(g, base=2.0, per_ms=0.5, seed=11):             # c(u->v) = base + per_ms*latency + phi(u) - phi(v)
    phi = np.random.RandomState(seed).uniform(0, 12, g.n)
    arcs = [(u, v, base + per_ms*w + phi[u] - phi[v]) for u, v, w in g.edges] + [(v, u, base + per_ms*w + phi[v] - phi[u]) for
      ↪ u, v, w in g.edges]
    return Graph(g.nodes, arcs, directed=True), phi
```

Code cell 4: placement (assignment, greedy, local search, MILP, solver)

```
# ---------------- placement: assignment, greedy, local search, exact MILP ----------------
class Inst: pass
def make_inst(g, K, budget, core_frac=.45, edge_frac=.25, core_cost=30, edge_cost=12):
    I = Inst(); I.g, I.K, I.budget = g, K, budget; I.names = [a['name'] for a in g.nodes]; I.D = apsp(g).dist
    I.dem = np.array([a['demand'] for a in g.nodes], float); core = np.array([a['tier'] == 'core' for a in g.nodes])
    I.capu = np.where(core, core_frac, edge_frac) * I.dem.sum(); I.cost = np.where(core, core_cost, edge_cost).astype(float)
    I.cand = list(range(g.n)); return I

def assign(I, R):                                                      # demand-first, nearest replica that still has room
    rem, a, un = [I.capu[j] for j in R], -np.ones(len(I.dem), int), 0.0
    for v in sorted(range(len(I.dem)), key=lambda v: -I.dem[v]):
        j = next((j for j in sorted(range(len(R)), key=lambda j: I.D[v, R[j]]) if rem[j] >= I.dem[v] - 1e-9), None)
        if j is None: un += I.dem[v]; continue
        rem[j] -= I.dem[v]; a[v] = j
    load, s = np.zeros(len(R)), 0.0
    for v in range(len(a)):
        if a[v] >= 0: load[a[v]] += I.dem[v]; s += I.dem[v] * I.D[v, R[a[v]]]
    served = I.dem.sum() - un
    return dict(R=list(R), a=a, load=load, unplaced=un, L=s/served if served else math.inf)
obj = lambda I, r: r['L'] + 1000*r['unplaced']/I.dem.sum()
def p95(I, r):
    c, tot = 0, I.dem.sum() - r['unplaced']
    for l, d in sorted((I.D[v, r['R'][r['a'][v]]], I.dem[v]) for v in range(len(I.dem)) if r['a'][v] >= 0):
```

```
        c += d
        if c >= .95*tot: return l

def greedy(I):                                                  # add the site that lowers the objective most
    R = []
    for _ in range(I.K):
        best = min(((obj(I, assign(I, R+[c])), c) for c in I.cand if c not in R and I.cost[R+[c]].sum() <= I.budget),
          ↪ default=None)
        if best is None: break
        R.append(best[1])
    return assign(I, R)
def local_search(I, r):                                         # swap one site at a time while the objective improves
    R, cur, imp = list(r['R']), obj(I, r), True
    while imp:
        imp = False
        for i in range(len(R)):
            for c in I.cand:
                T = R[:i] + [c] + R[i+1:]
                if c in R or I.cost[T].sum() > I.budget: continue
                o = obj(I, assign(I, T))
                if o < cur - 1e-9: R, cur, imp = T, o, True; break
            if imp: break
    return assign(I, R)
def multistart(I, starts=30, seed=7):                           # many random starts, keep the best local optimum
    rg, best = random.Random(seed), local_search(I, greedy(I))
    for _ in range(starts):
        R = rg.sample(I.cand, I.K)
        if I.cost[R].sum() > I.budget: continue
        r = local_search(I, assign(I, R))
        if obj(I, r) < obj(I, best) - 1e-9: best = r
    return best
def rand_base(I, trials=200, seed=7):
    rg, res = random.Random(seed), []
    while len(res) < trials:
        R = rg.sample(I.cand, I.K)
        if I.cost[R].sum() <= I.budget:
            r = assign(I, R)
            if r['unplaced'] == 0: res.append((r['L'], p95(I, r)))
    return np.mean([x[0] for x in res]), np.mean([x[1] for x in res])

def milp_opt(I, relax=False):                                   # exact assignment MILP (HiGHS); relax=True gives the LP
  ↪ lower bound
    N = len(I.dem); nv = N + N*N; rows, lo, hi = [], [], []
    c = np.concatenate([np.zeros(N), (I.dem[:, None] * I.D / I.dem.sum()).ravel()])
    for i in range(N): r = np.zeros(nv); r[N+i*N:N+(i+1)*N] = 1; rows.append(r); lo.append(1); hi.append(1)          # each
      ↪ node served once
    for j in range(N): r = np.zeros(nv); r[N+j::N] = I.dem; r[j] = -I.capu[j]; rows.append(r); lo.append(-np.inf); hi.append(0)
      ↪   # capacity
    for i in range(N):
        for j in range(N): r = np.zeros(nv); r[N+i*N+j] = 1; r[j] = -1; rows.append(r); lo.append(-np.inf); hi.append(0)   #
          ↪ x_ij <= y_j
    r = np.zeros(nv); r[:N] = 1; rows.append(r); lo.append(0); hi.append(I.K)
    r = np.zeros(nv); r[:N] = I.cost; rows.append(r); lo.append(0); hi.append(I.budget)
    res = milp(c, constraints=LinearConstraint(np.array(rows), lo, hi), integrality=np.zeros(nv) if relax else np.ones(nv),
      ↪ bounds=Bounds(0, 1), options={'time_limit': 60})
    if res.x is None: return math.nan, None, None
    if relax: return res.fun, None, None
    y = np.where(res.x[:N] > .5)[0].tolist(); X = res.x[N:].reshape(N, N)
    return res.fun, y, np.array([int(np.argmax(X[v, y])) for v in range(N)])      # sites, and the MILP's own optimal assignment

def pack(I, R, a):                                              # wrap an explicit assignment as a result dict
    load = np.zeros(len(R))
    for v in range(len(a)): load[a[v]] += I.dem[v]
    return dict(R=list(R), a=a, load=load, unplaced=0.0, L=sum(I.dem[v] * I.D[v, R[a[v]]] for v in range(len(a))) / I.dem.sum())

def solve(I):
    out, t = [], time.perf_counter()
    def add(name, kind, r=None, mean=None, p=None):
        out.append(dict(name=name, kind=kind, mean_ms=round(mean if mean is not None else r['L'], 3), p95_ms=round(p if p is
          ↪ not None else p95(I, r), 3),
                        runtime_ms=round((time.perf_counter()-t)*1e3, 1), r=r))
    m, p = rand_base(I); add('Random (mean of 200)', 'Heuristic', mean=m, p=p); t = time.perf_counter()
    r = greedy(I); add('Greedy', 'Heuristic', r); t = time.perf_counter()
    r = local_search(I, greedy(I)); add('Greedy + local search', 'Heuristic', r); t = time.perf_counter()
    r = multistart(I); add('Multi-start local search', 'Heuristic', r); t = time.perf_counter()
    L, R, a = milp_opt(I); add('Exact MILP', 'Exact', pack(I, R, a)); opt = out[-1]['mean_ms']
    for s in out: s['gap_pct'] = round(100*(s['mean_ms'] - opt)/opt, 2)
    best = out[-1]; r = best['r']; rd = out[0]; gr = out[1]
    names = [I.names[i] for i in r['R']]
    rec = [f"Host the {I.K} replicas at {', '.join(names[:-1])} and {names[-1]}.",
           f"Demand-weighted latency {best['mean_ms']} ms: {100*(rd['mean_ms']-best['mean_ms'])/rd['mean_ms']:.0f}% below a
             ↪ random placement ({rd['mean_ms']} ms) and {100*(gr['mean_ms']-best['mean_ms'])/gr['mean_ms']:.0f}% below plain
             ↪ greedy ({gr['mean_ms']} ms).",
           f"95% of requests are served within {best['p95_ms']} ms. Hosting cost {I.cost[r['R']].sum():.0f} of the
             ↪ {I.budget:.0f} budget.",
           'Busiest replica: ' + max(((I.names[s], 100*l/I.capu[s]) for s, l in zip(r['R'], r['load'])), key=lambda x: x[1])[0]
             ↪ + f" at {max(100*l/I.capu[s] for s, l in zip(r['R'], r['load'])):.0f}% of its capacity."]
    return dict(strategies=out, best=best['name'], recommendations=rec)
```

1. The campus network

Theory. A network is modelled as a weighted undirected graph: sites are vertices and links are edges weighted by latency. Latency between two sites follows a simple physical model, one-way:

$$\text{latency}(u, v) \;=\; 1.3 \cdot \frac{\text{km}(u, v)}{200 \text{ km/ms}} \;+\; 0.3 \text{ ms}$$

Light in fibre travels at roughly 200 km per millisecond; the factor 1.3 accounts for cables not following straight lines, and 0.3 ms is switching delay. On a campus the distances are a few hundred metres, so latency is dominated by the 0.3 ms switching term and every value is below about 1.3 ms.

Building the links. geo_graph connects every site to its three nearest neighbours, then adds the minimum spanning tree of the complete graph. The MST guarantees the result is **connected**, so every pair of sites has some route. Density is the share of possible links that exist: m / (n(n-1)/2).

> **Caveat.** The 12 campus sites and their positions are an illustrative layout, not the official Woxsen map; replace the CAMPUS list with real coordinates to match your report.

Code cell 6: campus graph

```
w = campus_graph()
print(w.summary())
pd.DataFrame([{'site': a['name'], 'tier': a['tier'], 'cpu': a['cpu']} for a in w.nodes]).head(12)
```

Output

```
12 nodes, 23 links, density 0.348
```

Output

	site	tier	cpu
0	Main Entrance (gate)	edge	32
1	Admin Block	core	64
2	Library	edge	32
3	Academic Block A	edge	32
4	Academic Block B	edge	32
5	Data Centre	core	64
6	Auditorium	edge	32
7	Student Centre	edge	32
8	Innovation Lab	edge	32
9	Hostel North	edge	32
10	Hostel South	edge	32
11	Sports Complex	edge	32

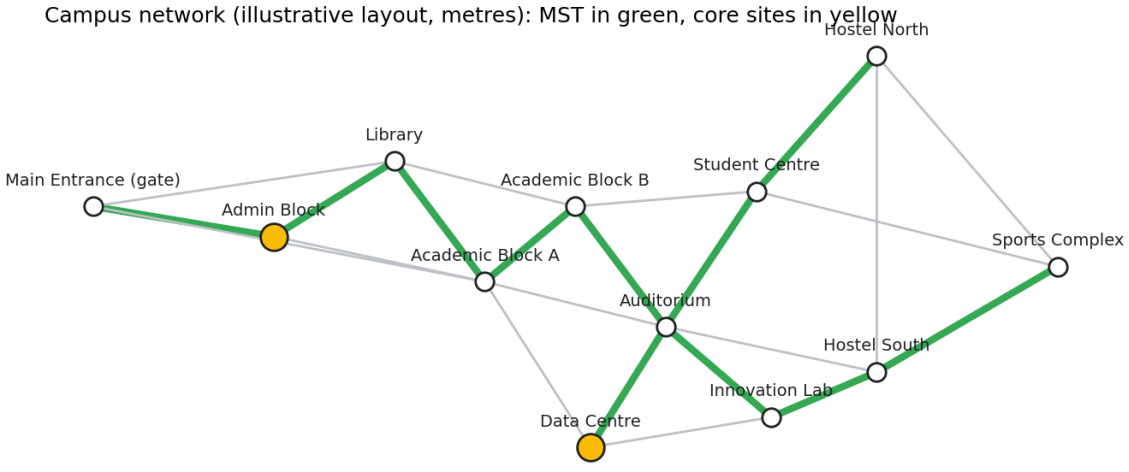


Figure 3. The campus graph. Green links form the MST; core sites (Admin Block, Data Centre) are yellow.

2. Backbone: Prim and Kruskal agree

Theory. A **minimum spanning tree** connects all sites using the cheapest possible set of links, with no cycles. Both algorithms rely on one fact:

> **Cut property.** For any split of the sites into two groups, the cheapest link crossing the split belongs to some MST. *Proof sketch:* if an MST T lacks that link e, adding e to T creates a cycle, and that cycle must cross the split at another link f with weight at least w(e). Swapping f for e gives a spanning tree that is no heavier.

Kruskal sorts all links by weight and accepts each one unless it would close a cycle (checked with union-find). Time O(E log E) = O(E log V). **Prim** grows a single tree outward, always adding the cheapest link that leaves the tree, using a min-heap. Time O(E log V) with a binary heap. Different order, same total weight.

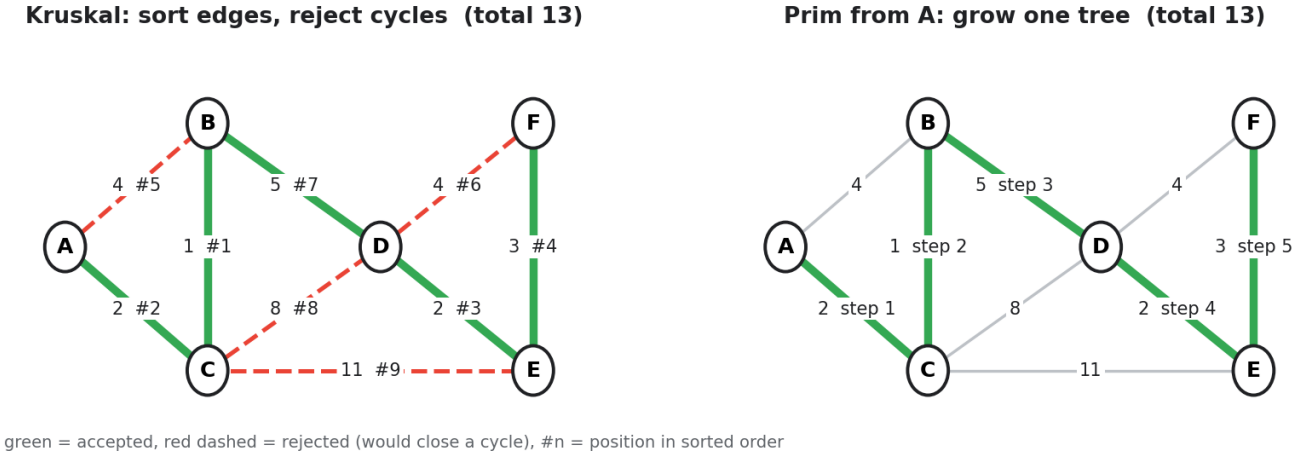


Figure 4. Kruskal and Prim on the same six-site graph reach the same total weight (13) by different routes.

Algorithm	Data structure	Time	Notes
Kruskal	sort + union-find	O(E log E)	Good for sparse graphs; edge list
Prim (lazy)	heapq with stale entries	O(E log V)	Simple; fastest in Python in practice
Prim (eager)	indexed heap, decrease-key	O(E log V)	No duplicates in the heap
Prim (Fibonacci)	Fibonacci heap	O(E + V log V)	Best bound; slow constants (in the full project)

Code cell 8: Prim (both heaps) and Kruskal

```
for heap in ['lazy', 'eager']:
    t = time.perf_counter(); r = prim(w, heap=heap)
    print(f'Prim ({heap:5s} heap) weight = {r.total_weight:.4f} ms, edges = {len(r.edges)}, {(time.perf_counter()-t)*1e3:.2f}
      ↪ ms')
k = kruskal(w)
print(f'Kruskal            weight = {k.total_weight:.4f} ms, edges = {len(k.edges)}')
print('all three equal:', math.isclose(k.total_weight, prim(w).total_weight) and math.isclose(k.total_weight, prim(w,
  ↪ "eager").total_weight))
```

Output

```
Prim (lazy  heap) weight = 3.3072 ms, edges = 11, 0.04 ms
Prim (eager heap) weight = 3.3072 ms, edges = 11, 0.08 ms
Kruskal            weight = 3.3072 ms, edges = 11
all three equal: True
```

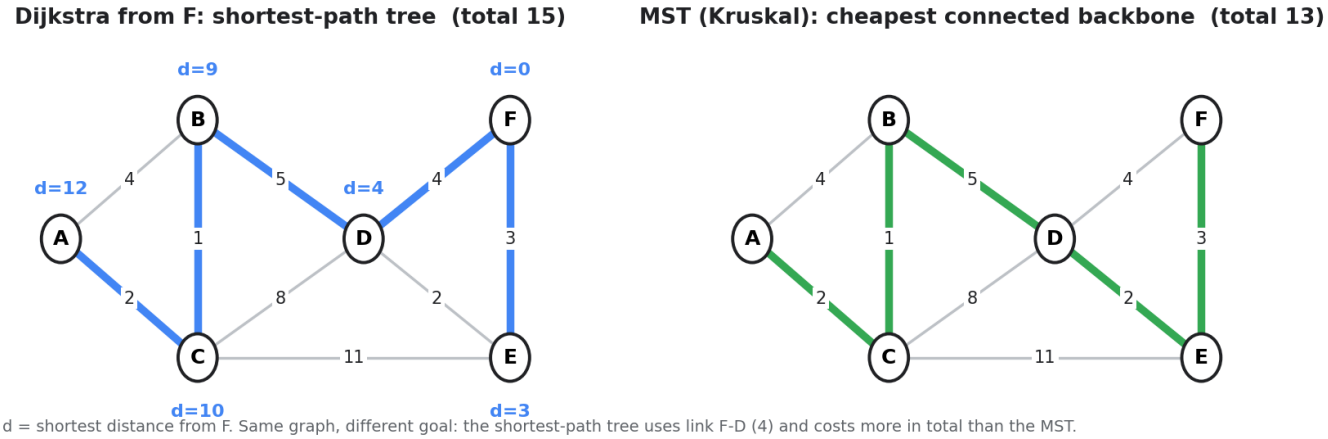
The three totals are identical, which is the practical check of the cut property: two different algorithms cannot disagree on the optimum.

3. Shortest paths from the Main Entrance (Dijkstra)

Theory. Dijkstra finds the shortest distance from one source to every site. It keeps a priority queue of tentative distances and repeatedly removes the closest unfinished site. Removing it is safe **because no link has negative weight**: any other route to that site would have to pass through a farther site and could only be longer. For each neighbour it then *relaxes* the link: if going through the current site is shorter, update the neighbour.

With a binary heap the time is O((V + E) log V). This implementation pushes new entries instead of decreasing keys, and discards stale ones when popped (*lazy deletion*). The `prev` map lets any shortest route be rebuilt hop by hop, which is how later sections "show the work".

Shortest path is not the MST. The MST minimises the total weight of the backbone; the shortest-path tree minimises each individual route from one source. They are different trees, as the diagram shows.



d = shortest distance from F. Same graph, different goal: the shortest-path tree uses link F-D (4) and costs more in total than the MST.

Figure 5. Same graph, two goals. The shortest-path tree from F uses link F-D (weight 4) and has total 15; the MST has total 13.

Code cell 10: Dijkstra from the Main Entrance

```
names = [a['name'] for a in w.nodes]
d = dijkstra(w, 0)
pd.DataFrame({'site': names, 'latency_ms': np.round(d.dist, 4), 'path': [' > '.join(names[x].split(' (')[0] for x in
  ↪ d.path_to(t)) for t in range(len(names))]}).sort_values('latency_ms').tail(6)
```

Output

	site	latency_ms	path
6	Auditorium	0.6025	Main Entrance > Academic Block A > Auditorium
7	Student Centre	0.9029	Main Entrance > Library > Academic Block B > S...
8	Innovation Lab	0.9031	Main Entrance > Academic Block A > Auditorium ...
10	Hostel South	0.9034	Main Entrance > Academic Block A > Auditorium ...
9	Hostel North	1.2037	Main Entrance > Library > Academic Block B > S...
11	Sports Complex	1.2042	Main Entrance > Library > Academic Block B > S...

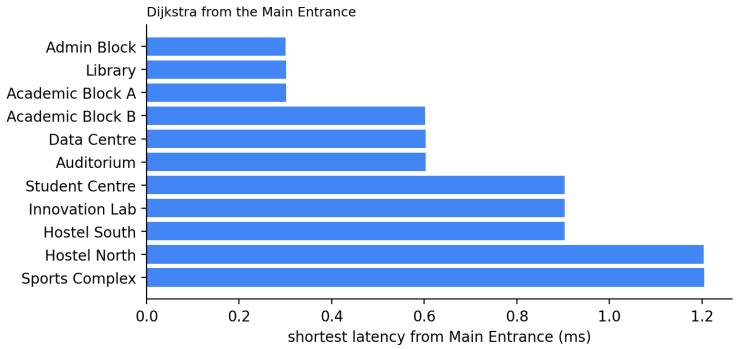


Figure 6. Shortest latency from the Main Entrance to each campus site.

4. Negative transfer costs: Bellman-Ford and Johnson

Theory. Moving data between sites has a cost that depends on distance and on each site's **price level** φ (for example storage or egress pricing). The transfer cost of an arc is:

$$c(u \to v) \;=\; \text{base} + \text{per_ms} \cdot \ell_{uv} + \varphi(u) - \varphi(v)$$

$$\sum_{(u \to v) \,\in\, \text{cycle}} c(u \to v) \;=\; \sum_{\text{cycle}} (\text{base} + \text{per_ms} \cdot \ell_{uv}) \;\geq\; 0$$

If data moves from an expensive site to a cheap one, the φ terms can outweigh the base cost and the arc becomes a **rebate** (negative). Dijkstra is then unsafe, which the notebook demonstrates by catching the error it raises.

Why a negative cycle can never appear. Around any cycle the φ terms telescope: each site is entered once and left once, so every φ(u) is added and subtracted exactly once and the total φ contribution is zero. What remains is the sum of base + per_ms · latency over the cycle, which is positive. So shortest transfer costs are always well defined.

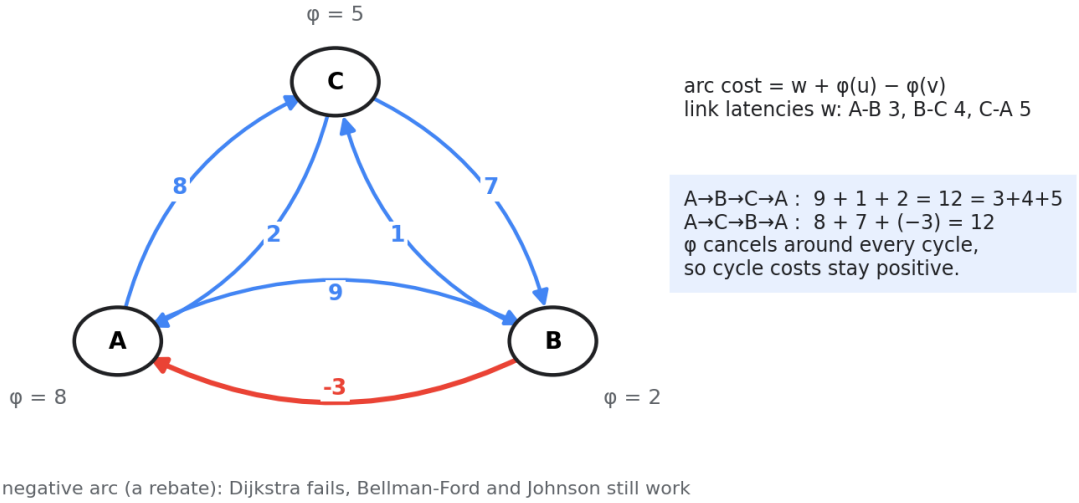


Figure 7. A triangle with price levels φ. Reverse arc B to A costs -3 (red), yet both directions of the cycle total 12 = 3 + 4 + 5.

Bellman-Ford handles negative arcs by relaxing every arc V - 1 times (a shortest path has at most V - 1 hops); one extra pass that still improves something proves a negative cycle. Time O(VE).

Johnson's algorithm reuses Bellman-Ford once to make Dijkstra applicable. Add a virtual source with zero-cost arcs to every site, run Bellman-Ford to get potentials h, then reweight:

$$w'(u,v) \;=\; w(u,v) + h(u) - h(v) \;\geq\; 0 \qquad\qquad d(u,v) \;=\; d'(u,v) - h(u) + h(v)$$

Every reweighted arc is non-negative because h satisfies the triangle inequality, and every route between two fixed sites changes by the same amount h(u) - h(v), so the shortest routes stay the same. After running Dijkstra from each source the offsets are undone. Time O(VE log V), better than Floyd-Warshall on sparse graphs.

Code cell 12: transfer costs, Bellman-Ford and Johnson

```
india = region_graph(INDIA)
tg, phi = transfer_cost_graph(india)
neg = sum(1 for u, v, c in tg.edges if c < 0)
print('directed arcs:', len(tg.edges), ' negative arcs:', neg)
try:
    dijkstra(tg, 0)
except ValueError as e:
```

```
    print('Dijkstra refuses:', str(e)[:80], '...')
bf = bellman_ford(tg, 0); jn = johnson(tg)
print('Bellman-Ford and Johnson agree:', np.allclose(bf.dist, jn[0]))
arc = {(u, v): c for u, v, c in tg.edges}                      # round trip 0 -> far city -> 0 is a cycle
p = dijkstra(india, 0).path_to(india.n - 1); cyc = p + p[::-1][1:]
with_phi = sum(arc[(a, b)] for a, b in zip(cyc, cyc[1:])); no_phi = sum(2.0 + 0.5 * dict(india.adj[a])[b] for a, b in zip(cyc,
  ↪ cyc[1:]))
print(f'round trip {india.nodes[0]["name"]} > {india.nodes[-1]["name"]} > back: cost with phi = {with_phi:.3f}, without phi =
  ↪ {no_phi:.3f}, phi cancels: {math.isclose(with_phi, no_phi)}')
```

Output

```
directed arcs: 94  negative arcs: 18
Dijkstra refuses: negative arc 0->8 (-6.22): Dijkstra needs w >= 0 ...
Bellman-Ford and Johnson agree: True
round trip Delhi > Woxsen University > back: cost with phi = 26.103, without phi = 26.103, phi cancels: True
```

5. All-pairs latency: the automatic choice

Theory. Placement needs dist(u, v) for every pair, the **latency matrix D**. Three algorithms can build it, and the best one depends on the graph:

$$D_{ij}^{(k)} = \min\left(D_{ij}^{(k-1)},\ D_{ik}^{(k-1)} + D_{kj}^{(k-1)}\right)$$

Floyd-Warshall grows the set of allowed stepping stones one site at a time: the best route from i to j either avoids site k or goes i to k to j. Three nested loops give $\Theta(V^3)$; the NumPy version vectorises the two inner loops into one outer minimum per k.



path 1-2-3-4 uses links 3 + 2 + 4 = 9, which beats the direct link of 10; red boxes mark entries that improved

Figure 8. Floyd-Warshall on four sites. Red boxes mark entries that improved when a new stepping stone was allowed.

Situation	Algorithm chosen	Time	Why
Negative arcs present	Johnson	O(VE log V)	Dijkstra alone is unsafe; Johnson repairs it
Dense graph (density > 0.25)	Floyd-Warshall (NumPy)	$\Theta(V^3)$	Simple, vectorised, density makes E close to V^2
Sparse graph	Repeated Dijkstra	O(VE log V)	Only real links are scanned

Code cell 14: automatic all-pairs choice

```
rg = random.Random(7); er_nodes = [dict(name=f'n{i}', demand=1, tier='edge') for i in range(100)]
er = Graph(er_nodes, [(i, i+1, round(rg.uniform(1, 20), 2)) for i in range(99)] + [(i, j, round(rg.uniform(1, 20), 2)) for i in
  ↪ range(100) for j in range(i+2, 100) if rg.random() < .05])
for name, g_ in [('india', india), ('nigeria', region_graph(NIGERIA)), ('er-100', er)]:
    t = time.perf_counter(); r = apsp(g_); ms = (time.perf_counter() - t) * 1e3
    print(f'{name:8s} n={g_.n:3d} density={g_.density():.3f} -> {r.algorithm:24s} {ms:7.2f} ms')
dense = Graph(er_nodes[:30], [(i, j, round(rg.uniform(1, 20), 2)) for i in range(30) for j in range(i+1, 30) if rg.random() <
  ↪ .6])
r = apsp(dense); print(f'dense-30 n= 30 density={dense.density():.3f} -> {r.algorithm:24s} (dense graph)')
print('Floyd-Warshall == Johnson == repeated Dijkstra on india:', np.allclose(floyd_warshall_numpy(india), johnson(india)) and
  ↪ np.allclose(johnson(india), np.array([dijkstra(india, s).dist for s in range(india.n)])))
```

Output

```
india    n= 25 density=0.157 -> repeated Dijkstra           0.45 ms
nigeria  n= 24 density=0.170 -> repeated Dijkstra           0.40 ms
er-100   n=100 density=0.069 -> repeated Dijkstra          11.94 ms
dense-30 n= 30 density=0.644 -> Floyd-Warshall (numpy)   (dense graph)
Floyd-Warshall == Johnson == repeated Dijkstra on india: True
```

The three sparse networks (India, Nigeria, the 100-node random graph) all select repeated Dijkstra, while a dense 30-node graph selects Floyd-Warshall. The last line confirms that Floyd-Warshall, Johnson and repeated Dijkstra produce the same matrix on the India network.

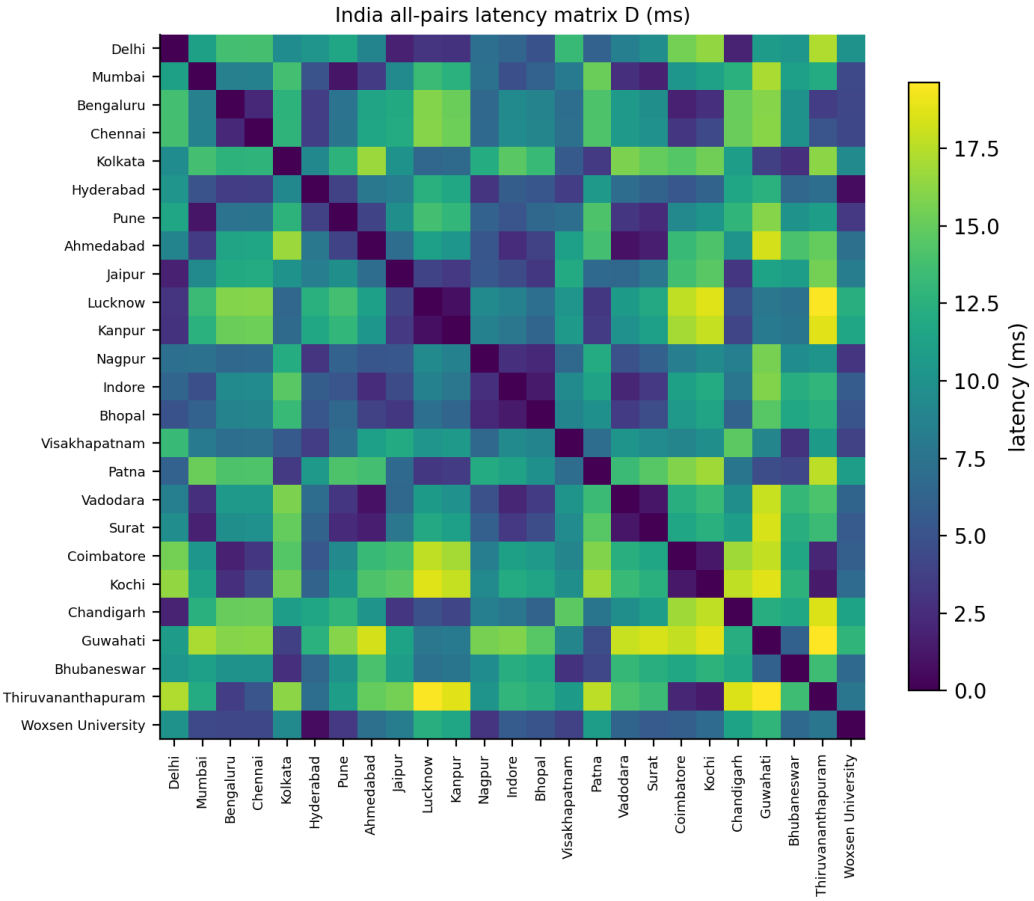


Figure 9. The India latency matrix D. Dark cells are close pairs; bright cells are far pairs such as Thiruvananthapuram to Guwahati.

6. Knapsack DP: choosing servers within a budget

Theory. Activating servers under a budget is a **0/1 knapsack**: each server is an item with a value (benefit) and a weight (cost); take each at most once to maximise total value without exceeding the budget W. Dynamic programming solves it exactly:

$$T[i][c] \;=\; \max(T[i-1][c],\;\; T[i-1][c-w_i] + v_i) \quad (\text{if } w_i \le c)$$

T[i][c] is the best value using only the first i items with capacity c: either skip item i, or take it and add its value to the best result for the remaining capacity. The answer is T[n][W], and walking backwards through the table recovers which items were chosen. Time and space Θ(nW), which is *pseudo-polynomial*: polynomial in the budget value, not in its number of digits.

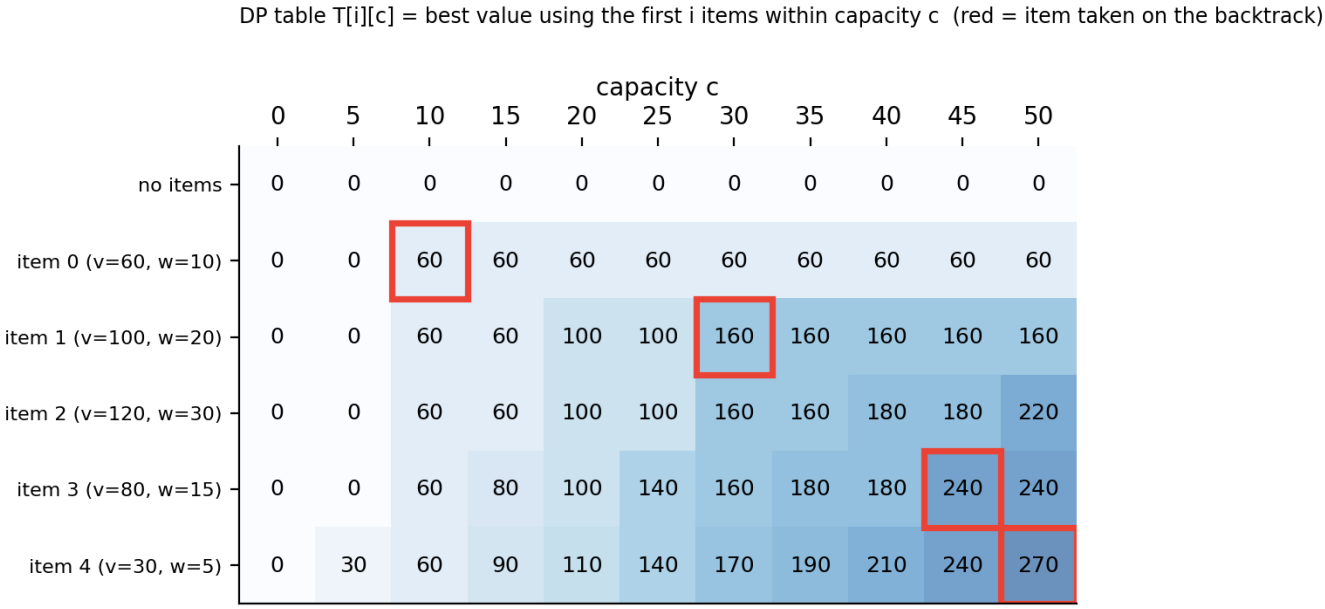


Figure 10. DP table for the notebook example. Red boxes trace the backtrack: items 4, 3, 1 and 0 are taken.

Code cell 16: 0/1 knapsack

```
vals, wts = [60, 100, 120, 80, 30], [10, 20, 30, 15, 5]
r = knapsack_01(vals, wts, 50)
print('chosen items', r.chosen, 'value', r.value, 'weight', r.weight)
```

Output

```
chosen items [0, 1, 3, 4] value 270 weight 50
```

The result (items 0, 1, 3, 4 with value 270 and weight exactly 50) is verified against a brute-force check of every subset in the self-checks section.

7. Full placement on India and Nigeria

Problem. Choose at most K replica sites to minimise L, subject to a hosting budget and per-site capacity. Capacity is measured in **demand units** (requests per second): core sites can serve 45% of total demand, edge sites 25%, so capacity genuinely constrains the choice. For India (K = 4, budget 90), four edge sites offer exactly 100% of total demand in aggregate, but because the demand of a single node cannot be split across replicas they cannot pack it all, which a self-check confirms.

Capacity-aware assignment

Given a set of replica sites, each node must be assigned to one replica. The notebook sorts nodes by demand, largest first, and sends each to its nearest replica that still has room. Demand that fits nowhere is reported as **unplaced** and heavily penalised (1000 / total demand per unit) so infeasible plans never win. This assignment is a heuristic: it is fast, but not always optimal.

Heuristics

Greedy adds, one at a time, the site that lowers the objective most. **Swap local search** then replaces one chosen site with an unchosen one whenever that improves the objective, until no single swap helps. It can get stuck in a *local optimum*: a plan where no single swap helps although swapping two sites together would. **Multi-start local search** repeats the search from many random starting plans and keeps the best, which escapes most such traps. **Random** is the baseline: the mean of 200 random feasible plans.

Exact optimum with a MILP

A **mixed-integer linear program** finds the true optimum. Binary variable y_j says site j hosts a replica; binary x_{ij} says node i is served from site j.

$$\text{minimise} \quad \sum_i \sum_j \frac{d_i \, \text{dist}_{ij}}{\sum_k d_k} x_{ij}$$

$$\text{subject to} \quad \sum_j x_{ij} = 1 \quad \text{for every node } i \qquad \text{(each node is served exactly once)}$$

$$\sum_i d_i \, x_{ij} \le \text{cap}_j \, y_j \qquad \text{(capacity, available only if site } j \text{ is open)}$$

$$x_{ij} \le y_j \qquad \text{(serve only from open sites; tightens the LP bound)}$$

$$\sum_j y_j \le K, \qquad \sum_j \text{cost}_j \, y_j \le B \qquad \text{(replica count and hosting budget)}$$

$$x_{ij},\, y_j \in \{0, 1\}$$

Because the solver (HiGHS) proves optimality, it is the yardstick for the heuristics. Setting the binaries to continuous values gives the **LP relaxation**, a lower bound that no integer plan can beat; the $x_{ij} <= y_j$ constraints make that bound much tighter. The heuristic quality is reported as the optimality gap:

$$\text{gap}\,(\%) \;=\; 100 \cdot \frac{L_{\text{heuristic}} - L^*}{L^*}$$

Design note. The "Exact MILP" row uses the MILP's *own* optimal assignment rather than re-running the greedy assignment on its sites. Mixing the two once let a heuristic appear to beat the exact optimum, and the self-checks caught it.

Code cell 18: solve every strategy for India and Nigeria

```
rows = []
for region, g_, K, budget in [('india', india, 4, 90), ('nigeria', region_graph(NIGERIA), 3, 75)]:
```

```
    inst = make_inst(g_, K, budget)
    out = solve(inst)
    for s in out['strategies']:
        rows.append({'region': region, 'strategy': s['name'], 'type': s['kind'], 'mean_ms': s['mean_ms'], 'p95_ms':
          ↪ s['p95_ms'], 'gap_%': s['gap_pct'], 'runtime_ms': s['runtime_ms']})
    if region == 'india':
        india_out, india_inst = out, inst
pd.DataFrame(rows).pivot(index='strategy', columns='region', values='mean_ms').sort_values('india')
```

Output

```
region                   india  nigeria
strategy
Exact MILP               2.061    1.362
Multi-start local search 2.191    1.477
Greedy                   2.969    1.477
Greedy + local search    2.969    1.477
Random (mean of 200)     4.512    2.454
```

Code cell 19: detailed India results

```
pd.DataFrame(rows)[lambda d: d.region == 'india'].drop(columns='region').set_index('strategy')
```

Output

strategy	type	mean_ms	p95_ms	gap_%	runtime_ms
Random (mean of 200)	Heuristic	4.512	11.473	118.92	41.9
Greedy	Heuristic	2.969	9.521	44.06	5.4
Greedy + local search	Heuristic	2.969	9.521	44.06	12.1
Multi-start local search	Heuristic	2.191	5.560	6.31	558.8
Exact MILP	Exact	2.061	4.757	0.00	64.1

Code cell 20: plain-language recommendations

```
for line in india_out['recommendations']:
    print('•', line)
```

Output

```
• Host the 4 replicas at Mumbai, Bengaluru, Kolkata and Jaipur.
• Demand-weighted latency 2.061 ms: 54% below a random placement (4.512 ms) and 31% below plain greedy (2.969 ms).
• 95% of requests are served within 4.757 ms. Hosting cost 84 of the 90 budget.
• Busiest replica: Jaipur at 98% of its capacity.
```

Reading the results. The exact plan reaches 2.06 ms. Multi-start local search is within about 6% of it, plain greedy is 44% worse, and a random placement is more than twice as slow (+119%). Local search starting from greedy did not improve on greedy here because it was trapped in a local optimum, which is exactly why the multi-start row exists.

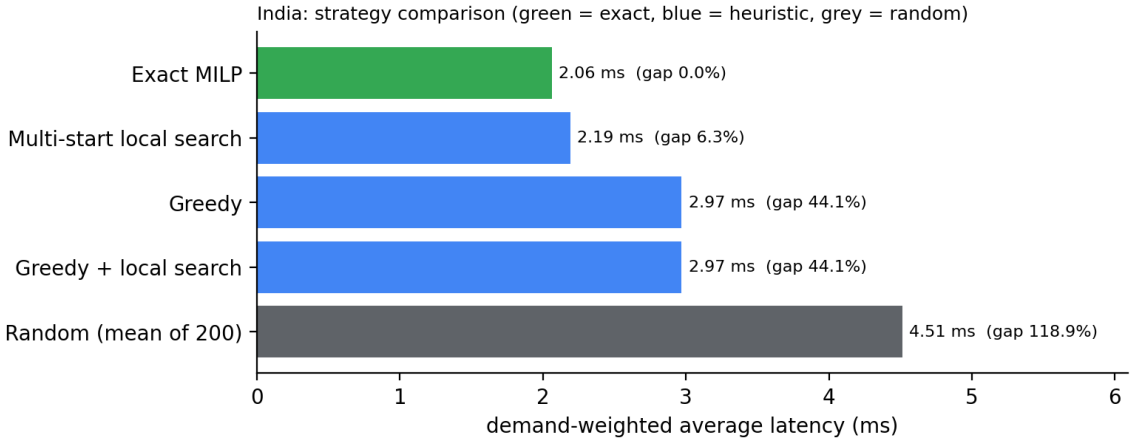


Figure 11. Mean latency by strategy for India, with the gap to the exact MILP optimum.

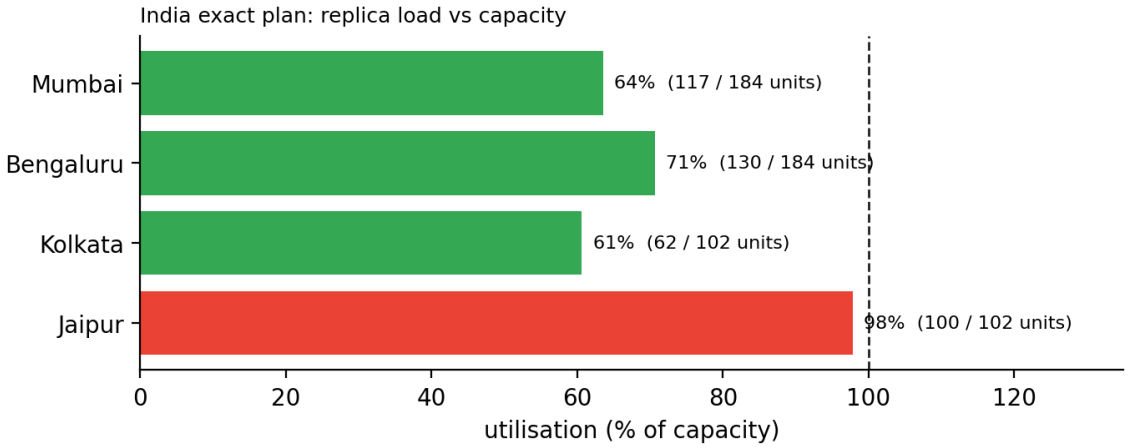


Figure 12. How full each chosen replica is. Jaipur runs at 98% of its capacity, so capacity is nearly binding there (hosting cost is 84 of the 90 budget).

8. The India plan on a map

The map draws every link in light blue, the shortest route from each city to its serving replica in darker blue, and the chosen replica sites as larger yellow nodes. The routes are rebuilt with Dijkstra's prev pointers, so each drawn route is exactly the path whose latency was counted in L.

Code cell 22: map of the optimal India plan

```
g = india_inst.g; best = next(s for s in india_out['strategies'] if s['name'] == india_out['best']); br = best['r']
hosts = set(br['R'])
fig, ax = plt.subplots(figsize=(7, 7))
for u, v, wt in g.edges:
    a, b = g.nodes[u], g.nodes[v]; ax.plot([a['x'], b['x']], [a['y'], b['y']], color=BLUE, alpha=.35, lw=1)
for v in range(g.n):
    if br['a'][v] >= 0 and br['R'][br['a'][v]] != v:
        p = dijkstra(g, v).path_to(br['R'][br['a'][v]]); ax.plot([g.nodes[i]['x'] for i in p], [g.nodes[i]['y'] for i in p],
          ↪ color='#1A73E8', lw=2, alpha=.5)
for i, a in enumerate(g.nodes):
    ax.scatter(a['x'], a['y'], s=70 if i in hosts else 25, c=YEL if i in hosts else 'white', edgecolors=INK, zorder=3)
    ax.annotate(a['name'], (a['x'] + .25, a['y'] + .15), fontsize=7, color=GREY)
ax.set_aspect(1.1); ax.axis('off'); ax.set_title(f"India: {best['name']} placement, mean {best['mean_ms']} ms")
plt.show()
```

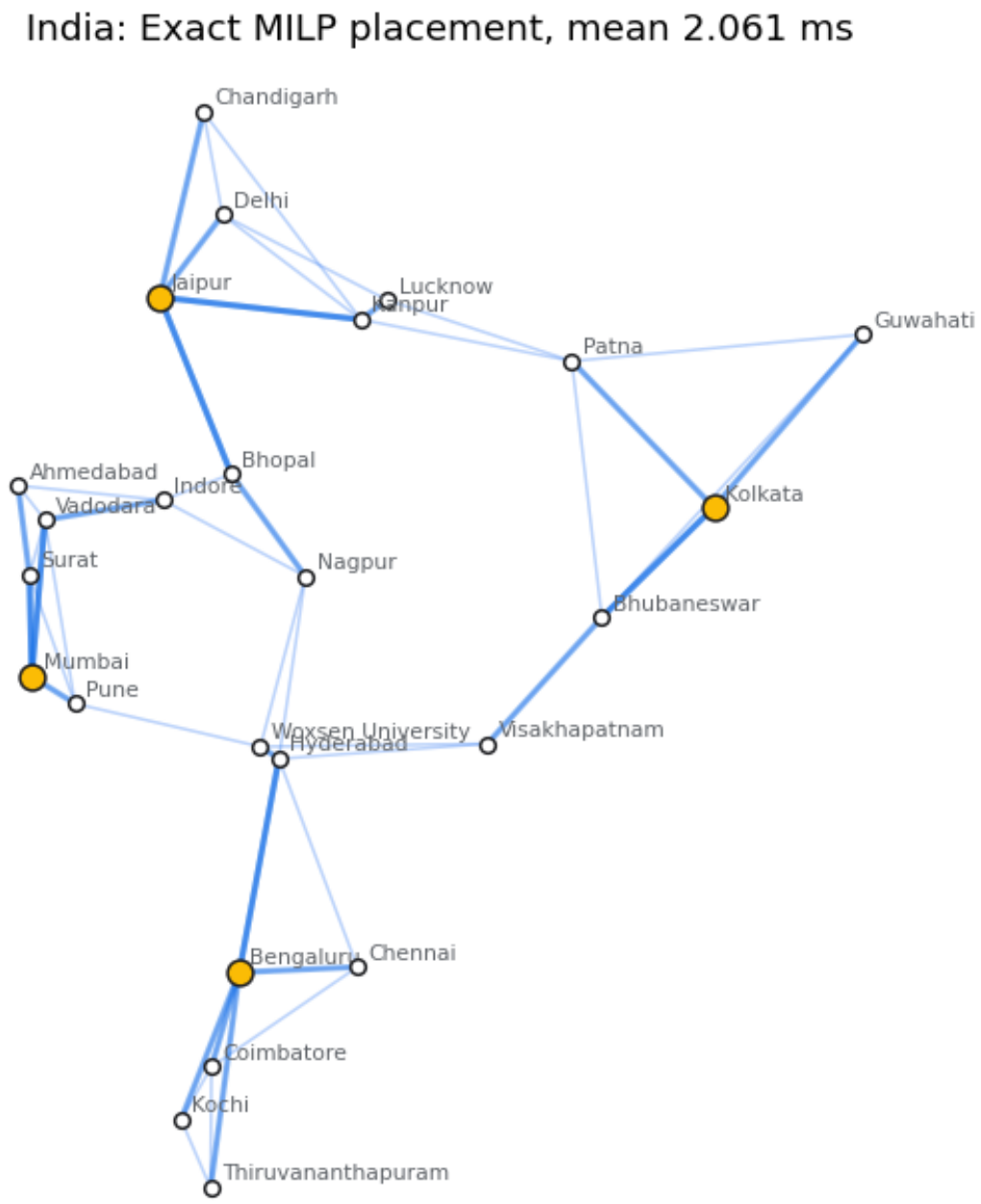


Figure 13. Figure produced by the cell above (rendered output of the notebook).

9. Experiment highlights

Design. A small version of the project's quality-versus-size study: random geographic networks of 10, 14, 18 and 22 nodes with three random seeds each (12 networks), K = 3, an unlimited budget and every site able to carry 45% of total demand. For each network the exact MILP gives the optimum, and each heuristic's gap to it is recorded.

Code cell 24: gap to the exact optimum

```
def random_inst(n, seed):
    rs = np.random.RandomState(seed)
    nodes = [dict(name=f'n{i}', x=rs.uniform(0, 10), y=rs.uniform(0, 10), demand=int(rs.randint(4, 20)), tier='edge', cpu=32)
      ↪ for i in range(n)]
    g_ = geo_graph(nodes, lambda a, b: math.hypot(a['x']-b['x'], a['y']-b['y']) * 150)
    I = make_inst(g_, 3, 1e9, edge_frac=.45, edge_cost=10); return I
res = []
for n_ in [10, 14, 18, 22]:
    for sd in range(3):
        I = random_inst(n_, sd); L, R, _ = milp_opt(I)
        for name, r in [('greedy', greedy(I)), ('greedy + local search', local_search(I, greedy(I))), ('multi-start local
          ↪ search', multistart(I, 20))]:
            res.append(dict(n=n_, seed=sd, strategy=name, gap_pct=100*(r['L']-L)/L))
Q = pd.DataFrame(res)
Q.groupby('strategy').gap_pct.agg(['mean', 'max']).round(2).sort_values('mean')
```

Output

strategy	mean	max
multi-start local search	1.08	9.00
greedy + local search	3.26	13.00
greedy	14.93	48.98

Takeaways. Multi-start local search averages about a 1% gap (worst case 9%), greedy plus local search about 3% (worst 13%), and plain greedy about 15% (worst 49%). More search effort buys better quality, and the cheapest method can be far from optimal when capacity is tight. With only 12 networks these figures are indicative, not statistically conclusive; the full experiment suite in the project uses more instances.

Code cell 25: gap versus network size

```
fig, ax = plt.subplots(figsize=(7, 3.8))
for (name, grp), col in zip(Q.groupby('strategy'), [BLUE, GRN, RED]):
    m = grp.groupby('n').gap_pct.mean(); ax.plot(m.index, m.values, 'o-', color=col, label=name)
ax.set_xlabel('network size n'); ax.set_ylabel('gap to exact optimum (%)'); ax.set_title('Heuristic quality vs size (mean of 3
  ↪ seeds)'); ax.legend(frameon=False); plt.show()
```

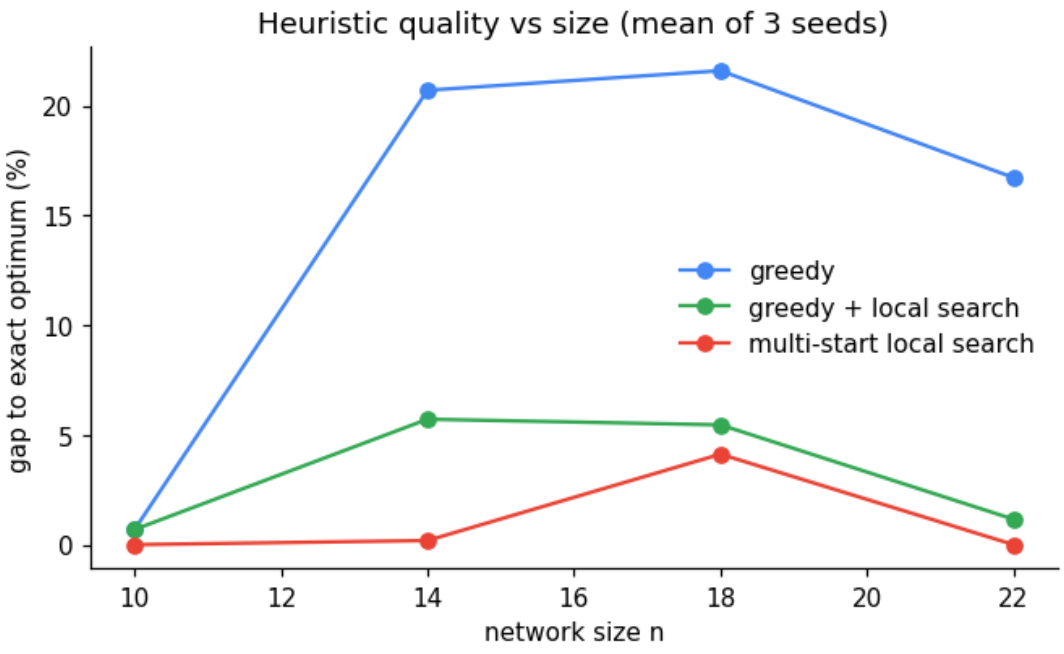


Figure 14. Figure produced by the cell above (rendered output of the notebook).

10. Self-checks

Every claim above is backed by an automatic check at the end of the notebook. All ten print PASS and each one asserts, so a failure stops the run.

Check	What it proves
Prim (both heaps) == Kruskal weight	Two MST algorithms agree (cut property)
MST has n - 1 edges	The backbone is a tree and spans every site
Floyd-Warshall == Johnson == Dijkstra	The latency matrix is right, three independent ways
Negative arcs exist but no negative cycle	Bellman-Ford and Johnson are really needed and always succeed
Knapsack DP == brute force	The DP returns the true optimum on the example
Capacity never exceeded	The exact plan respects every replica limit
Cost within budget	The exact plan respects the hosting budget
Exact <= every heuristic	The MILP row is a valid lower bound on latency
Per-node contributions sum to L	The headline number reconciles with the per-node numbers
Infeasible case detected	Four edge sites cannot pack all the (unsplittable) demand, and the code reports it

Code cell 27: self-checks

```
def check(name, ok): print(('PASS  ' if ok else 'FAIL  ') + name); assert ok, name
ex = next(s for s in india_out['strategies'] if s['name'] == 'Exact MILP'); er_ = ex['r']
check('Prim (both heaps) == Kruskal weight', math.isclose(prim(w).total_weight, kruskal(w).total_weight) and
  ↪ math.isclose(prim(w, 'eager').total_weight, kruskal(w).total_weight))
check('MST has n-1 edges', len(kruskal(india).edges) == india.n - 1)
check('Floyd-Warshall == Johnson == Dijkstra', np.allclose(floyd_warshall_numpy(india), johnson(india)))
check('Negative arcs exist but Bellman-Ford finds no negative cycle', neg > 0 and np.allclose(bf.dist, jn[0]))
import itertools
brute = max(sum(vals[i] for i in c) for k in range(len(vals)+1) for c in itertools.combinations(range(len(vals)), k) if
  ↪ sum(wts[i] for i in c) <= 50)
check('Knapsack DP == brute force over all subsets', knapsack_01(vals, wts, 50).value == brute)
check('Capacity never exceeded', all(er_['load'][j] <= india_inst.capu[s] + 1e-9 for j, s in enumerate(er_['R'])))
check('Cost within budget', india_inst.cost[er_['R']].sum() <= india_inst.budget)
check('Exact <= every heuristic', all(ex['mean_ms'] <= s['mean_ms'] + 1e-6 for s in india_out['strategies']))
check('Per-node contributions sum to L', math.isclose(sum(india_inst.dem[v]*india_inst.D[v, er_['R'][er_['a'][v]]] for v in
  ↪ range(india.n))/india_inst.dem.sum(), er_['L']))
check('Infeasible case detected (4 edge sites cannot hold all demand)', assign(india_inst, [i for i, a in
  ↪ enumerate(india.nodes) if a['tier'] == 'edge'][:4])['unplaced'] > 0)
```

Output

```
PASS  Prim (both heaps) == Kruskal weight
PASS  MST has n-1 edges
PASS  Floyd-Warshall == Johnson == Dijkstra
PASS  Negative arcs exist but Bellman-Ford finds no negative cycle
PASS  Knapsack DP == brute force over all subsets
PASS  Capacity never exceeded
PASS  Cost within budget
PASS  Exact <= every heuristic
PASS  Per-node contributions sum to L
PASS  Infeasible case detected (4 edge sites cannot hold all demand)
```

Appendix

A. Complexity summary

Stage	Algorithm	Time	Space
Latency matrix	Floyd-Warshall	$\Theta(V^3)$	$\Theta(V^2)$
Latency matrix	Johnson / repeated Dijkstra	$O(VE \log V)$	$\Theta(V^2)$
Backbone	Kruskal	$O(E \log E)$	$O(V + E)$
Backbone	Prim (binary heap)	$O(E \log V)$	$O(V + E)$
Routing	Dijkstra (heap)	$O(E \log V)$	$O(V)$
Routing	Bellman-Ford	$O(VE)$	$O(V)$
Servers	0/1 knapsack DP	$\Theta(nW)$	$\Theta(nW)$
Placement	Greedy	$O(K\ n)$ assignments	$O(n^2)$
Placement	Swap local search	$O(K\ n)$ per pass	$O(n^2)$
Optimality	MILP (HiGHS)	exponential worst case	$O(n^2)$ variables

B. Glossary

Term	Meaning
Replica	A copy of a service hosted at a site so nearby users can reach it quickly
Demand	Requests per second generated at a node
Capacity (demand units)	How much demand a site can serve, limited by CPU and memory
MST	Cheapest set of links connecting all sites without cycles
Relaxation	Updating a distance when a shorter route through another node is found
Potential (h or φ)	A per-node offset that changes arc costs without changing which paths are shortest
Local optimum	A plan no single swap can improve, though a better plan may exist
MILP	Optimisation with linear constraints and some integer variables, solved exactly
LP relaxation	The same problem with integers relaxed to continuous values; gives a lower bound
Optimality gap	How far a heuristic result is above the proven optimum, in percent
Jain fairness	Load balance index: 1 means perfectly equal loads

C. Where to change things

Edit `CAMPUS` to use the official campus coordinates. Edit `INDIA` and `NIGERIA` to change demands and tiers. Change K and budget in code cell 18, and `core_frac`, `edge_frac`, `core_cost` and `edge_cost` in `make_inst` to explore capacity and cost trade-offs. For networks beyond about 30 sites, skip the MILP and compare heuristics to the LP lower bound instead.